CHAPTER 1

딥러닝기초소개

성명 : 최승호
이메일 : jcn99250@naver.com

학습 목표

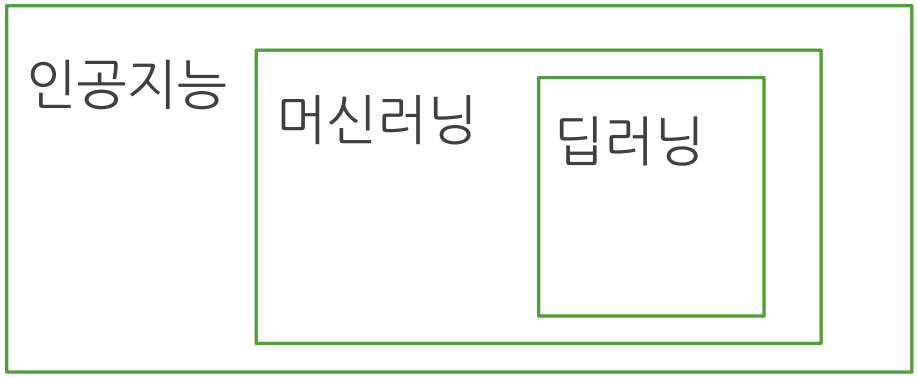
- 딥러닝 기초 소개에 대해서 이해합니다.
- 딥러닝 모델을 만들기 위해서 데이터 레이블링 방법에 대해서 알아보고자 합니다.

딥러닝 이란 (1/2)

- 인공지능 :
  - 기계가 경험을 통해 학습하고 새로운 입력 내용에 따라 기존 지식을 조정하며 사람과 같은 방식으로 과제를 수행할 수 있도록 지원하는 기술
- 머신러닝 :
  - 데이터를 분석하고 해당 데이터를 통해 학습한 후 정보를 바탕으로 결정 내리기 위해 학습한 내용을 적용하는 알고리즘
- 딥 러닝 :
  - 여러 층을 가진 인공신경망을 사용하여 학습하는 방법

딥러닝 이란 (2/2)

- 인공지능 종류:
  - 전문가 시스템, 규칙 기반 시스템
- 머신러닝 종류:
  - 결정 트리 , 선형 회귀, 퍼셉트론
- 딥러닝 종류 :
  - CNN, 강화학습, RNN



딥러닝 모델 학습 방법

- 학습 방법:
  - 지도 학습 :
    - 정답이 있는 학습 방법
  - 비지도 학습 :
    - 정답이 없는 학습 방법
  - 강화학습 :
    - 자신이 한 행동에 대해 보상을 받는 학습 방법

딥러닝 데이터 세트 구축 방법

- 딥러닝 데이터 세트 구축
  - Training data :
    - 딥 러닝 모델을 학습하기 위한 데이터
  - Validation data :
    - 학습된 딥 러닝 모델을 검증하기 위한 데이터
  - Test data :
    - 학습 완료된 딥러닝 모델을 테스트 하기위한 학습하지 않은 데이터
  - 데이터 구축 비율 : Training data 60 %, Validation data 20%, Test data 20%

딥러닝 개념

- 학습 방법
  - 초기 학습 데이터를 가지고 딥 러닝 모델 학습
- 검증 방법
  - 학습 된 딥 러닝 모델에 검증 데이터를 적용해 추가 학습 진행
- 테스트 방법
  - 검증데이터로 학습 완료된 딥 러닝 모델을 통해 학습하지 않은 테스트 데이터로 딥 러닝 모델 테스트

딥러닝 개념

- 딥러닝 기초 구성
  - Convolution layer
    - 특징 연산 하는 레이어
  - Normalization layer
    - Normalization 하는 레이어
    - 종류 : Batch normalization, Instance normalization
  - Pooling layer
    - 특징 추출 하는 레이어
    - 종류 : Max pooling, Average pooling

딥러닝 개념

- 딥러닝 기초 구성 (계속)
  - Activation function : 특징 활성화 하는 함수
    - 종류 : ReLU, eLU, Leaky ReLU 등등
  - Loss function :
  - 딥 러닝 모델 목적 로스
    - 종류 : Crossentropy, MSE Loss 등등
  - Optimizator :
  - 딥 러닝 모델 최적화 방법
    - 종류 : Adam, Nadam 등등

딥러닝 개념

- 딥러닝 기초 구성 (계속)
  - Regularization function :
    - 딥 러닝 모델 학습에 도움 되는 방법
    - 종류 : L1 regularization , L2 regularization
  - 학습 파라미터 :
    - 학습 설정에 필요한 파라미터 (Epoch, Seed, Learning rate)
    - Epoch : 학습 수, Seed : 초기 설정 값
    - Learning rate : 학습 률
    - Batch data : 학습 데이터에서 입력 데이터 사이즈

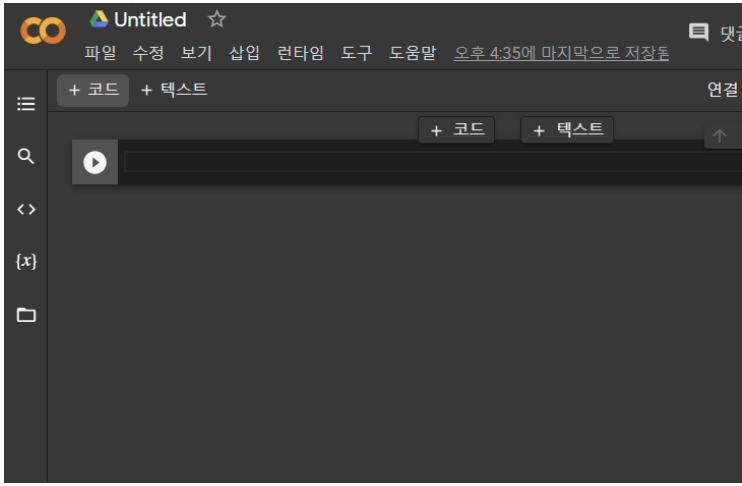
딥러닝 학습 판단

- 과적합
  - 정의 : 학습 데이터를 과하게 학습한 경우(암기)
  - Training Accuracy 는 증가, Test Accuracy 는 감소
- Underfitting
  - 정의 : 학습이 덜 되어 잇는 경우
  - Training Accuracy / Test Accuracy 가 둘다 감소
- 적절한 학습
  - 정의 : 학습이 적절하게 되어 있는 경우
  - Training Accuracy 보다 Test Accuracy 가 증가 되는 경우

딥러닝 프레임 워크 및 실습 환경 구축

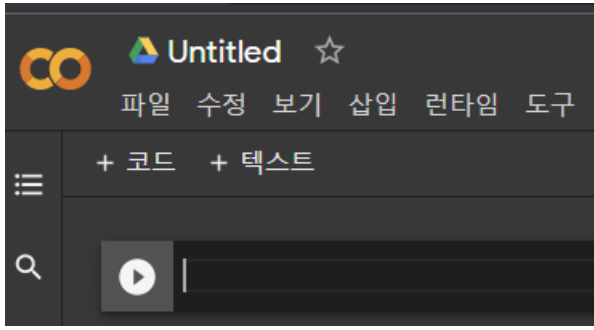
- 딥러닝 관련 Framework
  - 텐서플로 (*), 케라스 (*) , 파이토치

- 구글 코랩
  - 구글 계정 로그인
  - 검색 창에 “Google colab”검색
  - “Welcome To Colaboratory-Google Research” 접속
  - 새 노트 클릭
  - 작업 환경 구축



딥러닝 프레임 워크 및 실습 환경 구축

- 구글 코랩 (계속)
  - 실행 방법
    - 화살표 클릭 혹은 런타임 모두 실행

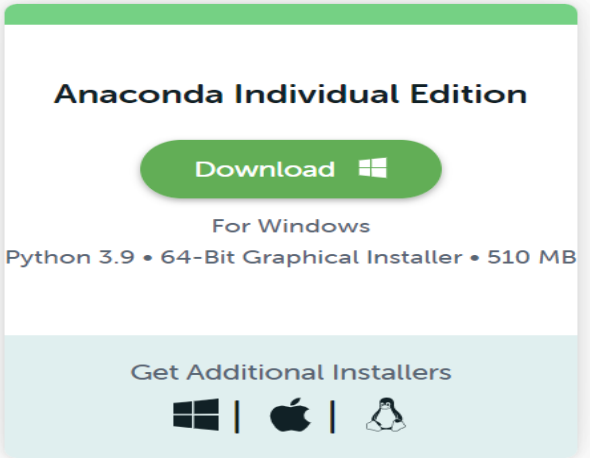


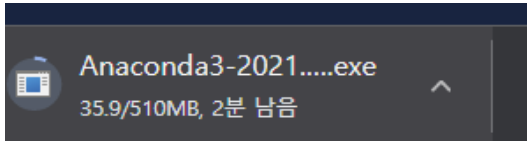
  - 코드 추가 방법
    - 코드 클릭

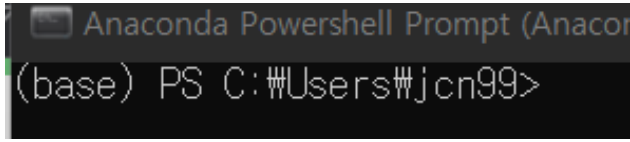
  - 파일 다운 방법
    - 파일 -> 다운로드 -> ipynb 파일 , py 파일

딥러닝 프레임 워크 및 실습 환경 구축

- 아나콘다 가상환경
  - 구글 에서 아나콘다 다운 검색
  - Individual Edition-Anaconda 클릭
  - Anaconda individual Edition 클릭
  - 우분투 혹은 윈도우 각자 환경에 맞게 다운
  - 다운 로드 파일 더블 클릭 실행해 설치
  - 설치 후 검색에서 Anaconda Powershell Prompt 클릭







딥러닝 프레임 워크 및 실습 환경 구축

- 아나콘다 가상환경 (계속)
  - 아나콘다 가상환경 만들기
    - conda create --name 가상환경이름 python= 버전
  - 가상환경 실행
    - conda activate py36
  - 가상환경 정보 확인
  - 가상환경 내에 필요한 패키지 설치

```
(base) PS C:₩Users₩jcn99> conda activate py36
(py36) PS C:₩Users₩jcn99>
```

```
(base) PS C:₩Users₩jcn99> conda info --envs
# conda environments:
#
base                  *  C:₩Users₩jcn99₩Anaconda3
bp_estimation            C:₩Users₩jcn99₩Anaconda3₩envs₩
bp_result_analysis       C:₩Users₩jcn99₩Anaconda3₩envs₩
feature_effect           C:₩Users₩jcn99₩Anaconda3₩envs₩
pcos                     C:₩Users₩jcn99₩Anaconda3₩envs₩
py36                     C:₩Users₩jcn99₩Anaconda3₩envs₩
recomdendation           C:₩Users₩jcn99₩Anaconda3₩envs₩
```

간단한 예제 구현 (1/5)

- 딥러닝 구현 예제:



https://colab.research.google.com/github/keras-team/keras-io/blob/master/examples/vision/ipynb/mnist_convnet.ipynb#scrollTo=oNTw8Pe4ohWe

간단한 예제 구현 (2/5)

- 딥러닝 구현 예제:

Prepare the data

```
[ ]  # Model / data parameters
     num_classes = 10
     input_shape = (28, 28, 1)

     # the data, split between train and test sets
     (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

     # Scale images to the [0, 1] range
     x_train = x_train.astype("float32") / 255
     x_test = x_test.astype("float32") / 255
     # Make sure images have shape (28, 28, 1)
     x_train = np.expand_dims(x_train, -1)
     x_test = np.expand_dims(x_test, -1)
     print("x_train shape:", x_train.shape)
     print(x_train.shape[0], "train samples")
     print(x_test.shape[0], "test samples")


     # convert class vectors to binary class matrices
     y_train = keras.utils.to_categorical(y_train, num_classes)
     y_test = keras.utils.to_categorical(y_test, num_classes)
```

https://colab.research.google.com/github/keras-team/keras-io/blob/master/examples/vision/ipynb/mnist_convnet.ipynb#scrollTo=oNTw8Pe4ohWe

간단한 예제 구현 (3/5)

- 딥러닝 구현 예제:

Build the model

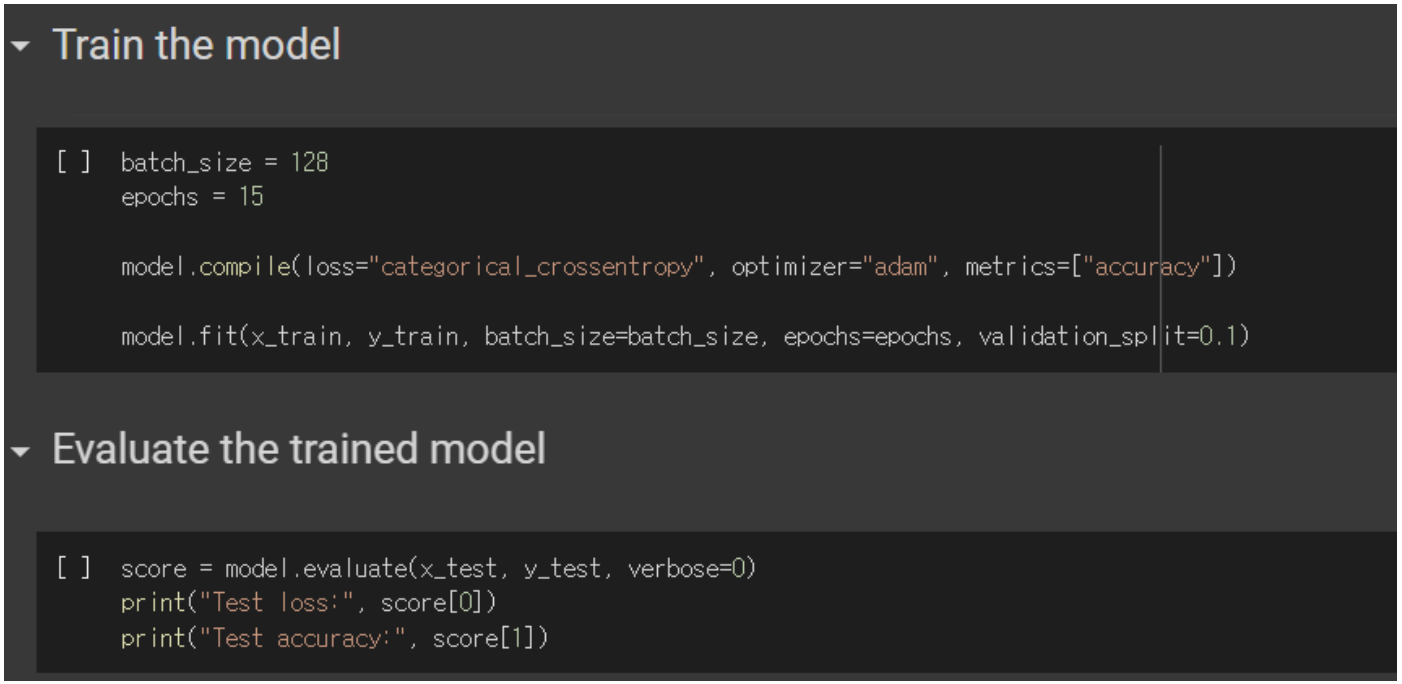
```
[ ]  model = keras.Sequential(
        [
            keras.Input(shape=input_shape),
            layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
            layers.MaxPooling2D(pool_size=(2, 2)),
            layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
            layers.MaxPooling2D(pool_size=(2, 2)),
            layers.Flatten(),
            layers.Dropout(0.5),
            layers.Dense(num_classes, activation="softmax"),
        ]
    )

    model.summary()
```

https://colab.resea
io/blob/master/examples/vision/ipynb/mnist_convnet.ipynb#scrollTo=oNTw8Pe4ohWe

간단한 예제 구현 (4/5)

- 딥러닝 구현 예제:



https://colab.research.google.com/github/keras-team/keras-io/blob/master/examples/vision/ipynb/mnist_convnet.ipynb#scrollTo=oNTw8Pe4ohWe

간단한 예제 구현 (5/5)

- 딥러닝 구현 예제:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11493376/11490434 [==============================] - 0s 0us/step
11501568/11490434 [==============================] - 0s 0us/step
x_train shape: (60000, 28, 28, 1)
60000 train samples
10000 test samples
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 32)        320

max_pooling2d (MaxPooling2D  (None, 13, 13, 32)        0
)

conv2d_1 (Conv2D)            (None, 11, 11, 64)        18496

max_pooling2d_1 (MaxPooling  (None, 5, 5, 64)          0
2D)

flatten (Flatten)            (None, 1600)              0

dropout (Dropout)            (None, 1600)              0

dense (Dense)                (None, 10)                16010

=================================================================
Total params: 34,826
Trainable params: 34,826
Non-trainable params: 0
_________________________________________________________________
```

```
Epoch 1/15
422/422 [==============================] - 40s 94ms/step - loss: 0.3785 - accuracy: 0.8864 - val_loss: 0.0872 - val_accuracy: 0.9763
Epoch 2/15
422/422 [==============================] - 38s 90ms/step - loss: 0.1122 - accuracy: 0.9659 - val_loss: 0.0566 - val_accuracy: 0.9862
Epoch 3/15
422/422 [==============================] - 37s 89ms/step - loss: 0.0805 - accuracy: 0.9753 - val_loss: 0.0448 - val_accuracy: 0.9877
Epoch 4/15
422/422 [==============================] - 39s 93ms/step - loss: 0.0698 - accuracy: 0.9782 - val_loss: 0.0471 - val_accuracy: 0.9865
Epoch 5/15
422/422 [==============================] - 37s 89ms/step - loss: 0.0593 - accuracy: 0.9819 - val_loss: 0.0361 - val_accuracy: 0.9880
Epoch 6/15
422/422 [==============================] - 37s 88ms/step - loss: 0.0549 - accuracy: 0.9823 - val_loss: 0.0363 - val_accuracy: 0.9883
Epoch 7/15
422/422 [==============================] - 37s 89ms/step - loss: 0.0501 - accuracy: 0.9847 - val_loss: 0.0349 - val_accuracy: 0.9897
Epoch 8/15
422/422 [==============================] - 37s 88ms/step - loss: 0.0469 - accuracy: 0.9851 - val_loss: 0.0327 - val_accuracy: 0.9893
Epoch 9/15
422/422 [==============================] - 37s 89ms/step - loss: 0.0439 - accuracy: 0.9860 - val_loss: 0.0312 - val_accuracy: 0.9908
Epoch 10/15
422/422 [==============================] - 38s 90ms/step - loss: 0.0424 - accuracy: 0.9861 - val_loss: 0.0309 - val_accuracy: 0.9915
Epoch 11/15
422/422 [==============================] - 38s 90ms/step - loss: 0.0394 - accuracy: 0.9870 - val_loss: 0.0272 - val_accuracy: 0.9915
Epoch 12/15
422/422 [==============================] - 38s 90ms/step - loss: 0.0377 - accuracy: 0.9885 - val_loss: 0.0284 - val_accuracy: 0.9910
Epoch 13/15
422/422 [==============================] - 38s 90ms/step - loss: 0.0358 - accuracy: 0.9880 - val_loss: 0.0314 - val_accuracy: 0.9897
Epoch 14/15
422/422 [==============================] - 38s 90ms/step - loss: 0.0337 - accuracy: 0.9890 - val_loss: 0.0285 - val_accuracy: 0.9920
Epoch 15/15
422/422 [==============================] - 38s 90ms/step - loss: 0.0311 - accuracy: 0.9895 - val_loss: 0.0291 - val_accuracy: 0.9927
Test loss: 0.023035062476992607
Test accuracy: 0.9919999837875366
```

https://colab.research.google.com/github/keras-team/keras-io/blob/master/examples/vision/ipynb/mnist_convnet.ipynb#scrollTo=oNTw8Pe4ohWe

어노테이션 툴 (1/14)

- 정의
  - 딥러닝 모델에 학습하기 위한 필요한 데이터를 만드는 가공하는 툴
- 장점:
  - 기존에 사용되어 오고 있는 유명한 Open source을 이용하여 Annotation 가능
- 단점:
  - 새로운 Annotation 방법을 적용해야 한다고 하면 개발하는 시간이 필요함
  - 사람이 Annotation tool에 기반하여 직접 레이블링을 진행해야 되기 때문에 시간이 많이 걸림
- 결과 예시
  - Output 형식 : json, xml, csv

어노테이션 툴 (2/14)

- 어노테이션 툴 예시:
  - Yolo_mark -> 물체 검출 관련 레이블링 (Object detection / Object tracking)
  - https://github.com/AlexeyAB/Yolomark
  - Labelme -> Segmentation 관련 레이블링 (semantic segmentation / instance segmentation)
  - https://github.com/wkentaro/labelme#windows
  - Video labeler
  - https://github.com/Cartucho/OpenLabeling

어노테이션 툴 (3/14)

- Yolo mark :
  - 물체 검출 레이블링에 주로 사용되는 툴

Windows & **Linux** GUI for marking bounded boxes of objects in images for training Yolo v3 and v2

- To compile on **Windows** open `yolo_mark.sln` in MSVS2013/2015, compile it **x64 & Release** and run the file: `x64/Release/yolo_mark.cmd`. Change paths in `yolo_mark.sln` to the OpenCV 2.x/3.x installed on your computer:
  - (right click on project) -> properties -> C/C++ -> General -> Additional Include Directories: `C:\opencv_3.0\opencv\build\include;`
  - (right click on project) -> properties -> Linker -> General -> Additional Library Directories: `C:\opencv_3.0\opencv\build\x64\vc14\lib;`
- To compile on **Linux** type in console 3 commands:

```
cmake .
make
./linux_mark.sh
```

어노테이션 툴 (4/14)

- Yolo mark (계속):
  - 설치 과정

Supported both: OpenCV 2.x and OpenCV 3.x

1. To test, simply run

- **on Windows:** `x64/Release/yolo_mark.cmd`
- **on Linux:** `./linux_mark.sh`

2. To use for labeling your custom images:

- delete all files from directory `x64/Release/data/img`
- put your `.jpg`-images to this directory `x64/Release/data/img`
- change numer of classes (objects for detection) in file `x64/Release/data/obj.data`: https://github.com/AlexeyAB/Yolo_mark/blob/master/x64/Release/data/obj.data#L1
- put names of objects, one for each line in file `x64/Release/data/obj.names`: https://github.com/AlexeyAB/Yolo_mark/blob/master/x64/Release/data/obj.names
- run file: `x64\Release\yolo_mark.cmd`

3. To training for your custom objects, you should change 2 lines in file `x64/Release/yolo-obj.cfg`:

- set number of classes (objects): https://github.com/AlexeyAB/Yolo_mark/blob/master/x64/Release/yolo-obj.cfg#L230
- set `filter`-value
  - For Yolov2 `(classes + 5)*5`: https://github.com/AlexeyAB/Yolo_mark/blob/master/x64/Release/yolo-obj.cfg#L224
  - For Yolov3 `(classes + 5)*3`

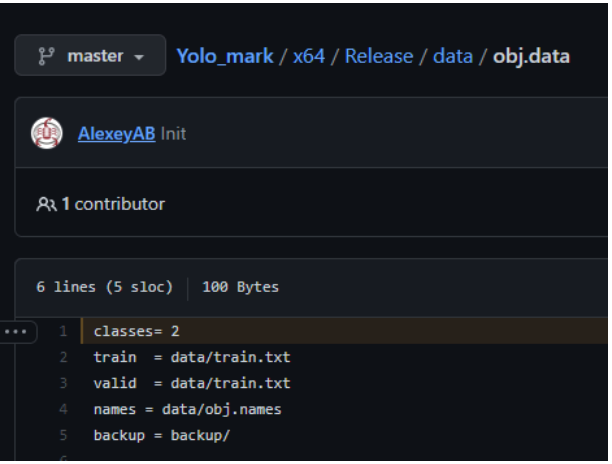
3.1 Download pre-trained weights for the convolutional layers (76 MB): http://pjreddie.com/media/files/darknet19_448.conv.23

3.2 Put files: `yolo-obj.cfg`, `data/train.txt`, `data/obj.names`, `data/obj.data`, `darknet19_448.conv.23` and directory `data/img` near with executable `darknet`-file, and start training: `darknet detector train data/obj.data yolo-obj.cfg darknet19_448.conv.23`

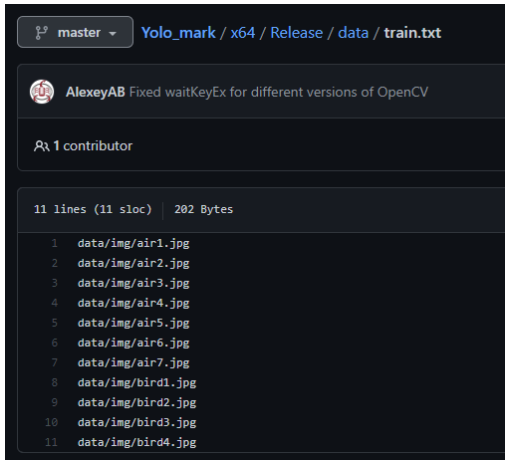
For a detailed description, see: https://github.com/AlexeyAB/darknet#how-to-train-to-detect-your-custom-objects

어노테이션 툴 (5/14)

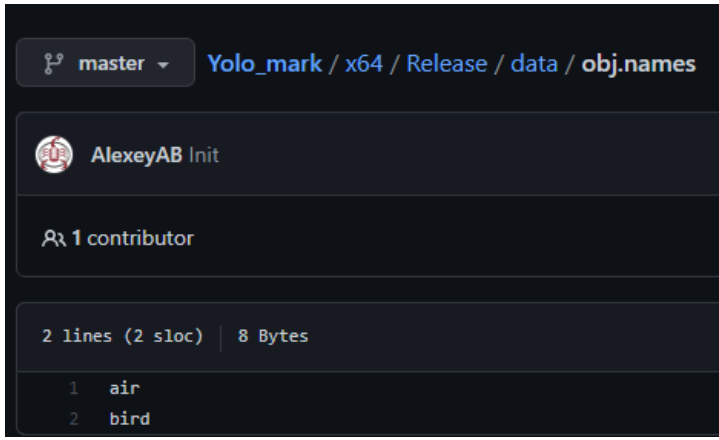
- Yolo mark :
  - 물체 검출 레이블링 적용시 변경해야 되는
  - 요소들
    - 클래스 이름 , 파일 목록, 전체 목록



전체 목록



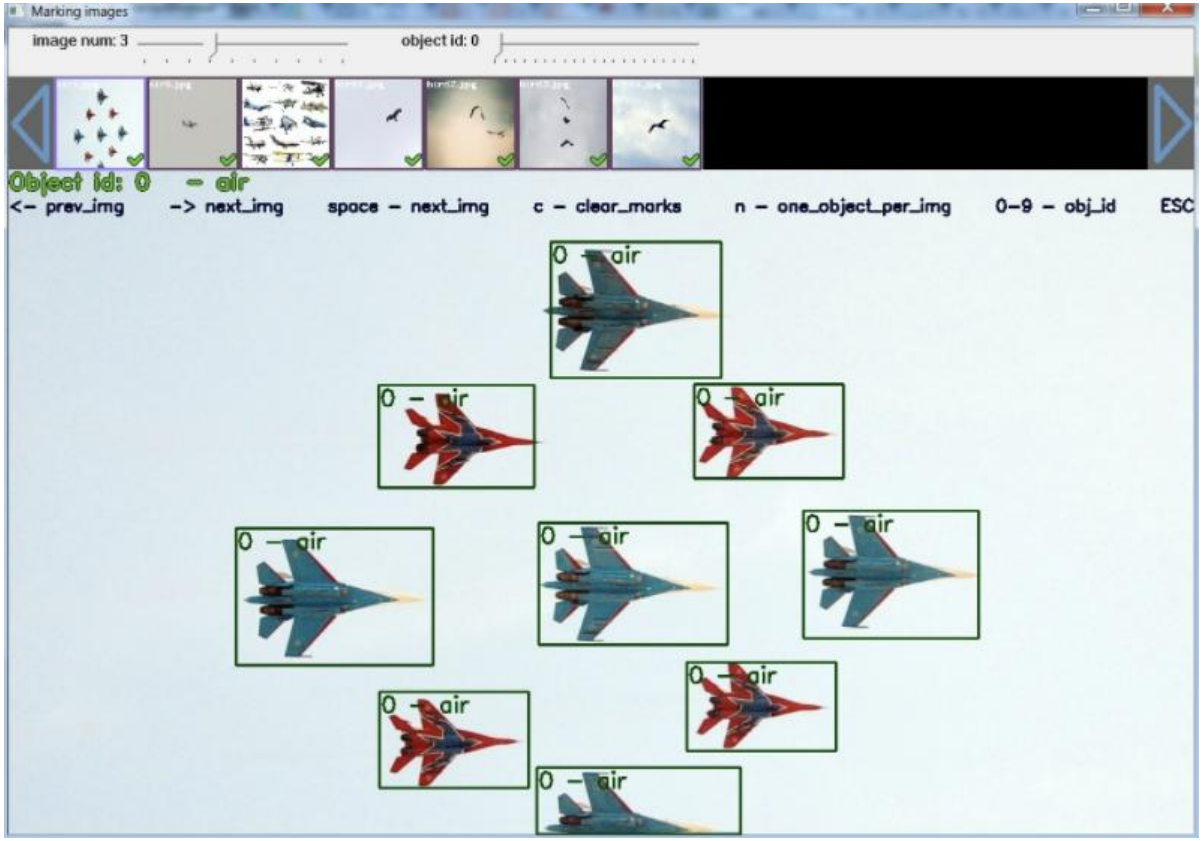
파일 목록



클래스 이름

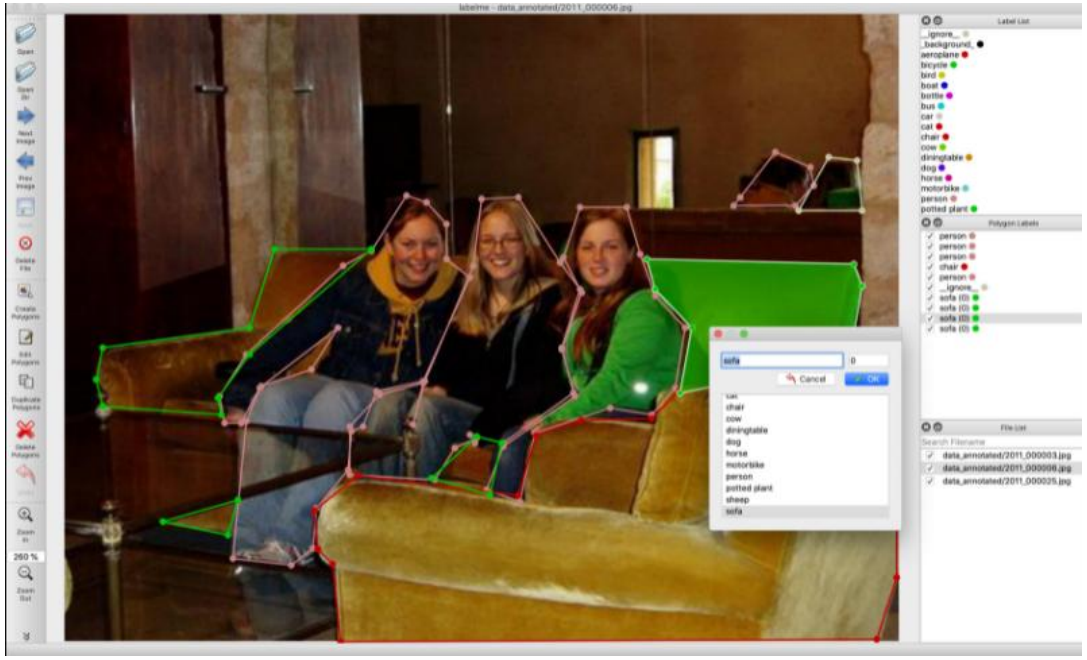
어노테이션 툴 (6/14)

- Yolo mark (계속):



어노테이션 툴 (7/14)

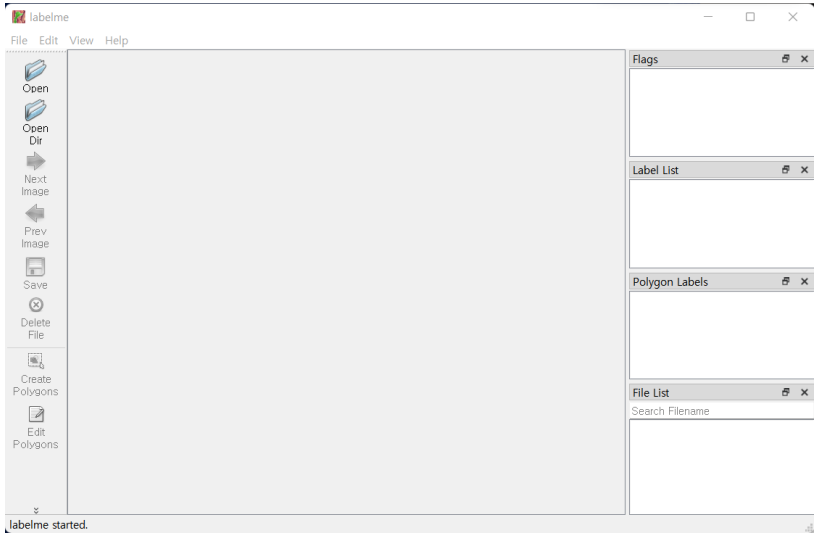
- 세그멘테이션 레이블링:
  - Labelme



어노테이션 툴 (8/14)

- 세그멘테이션 레이블링:
  - 설치 방법 : pip install labelme
  - 실행 방법 :

```
by36) PS C:₩Users₩jcn99> labelme
```



초기화면

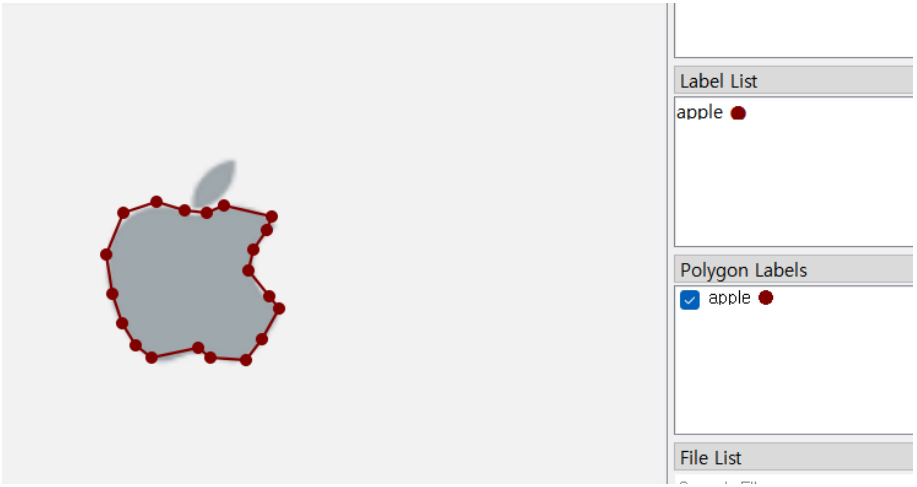


입력 이미지

어노테이션 툴 (9/14)

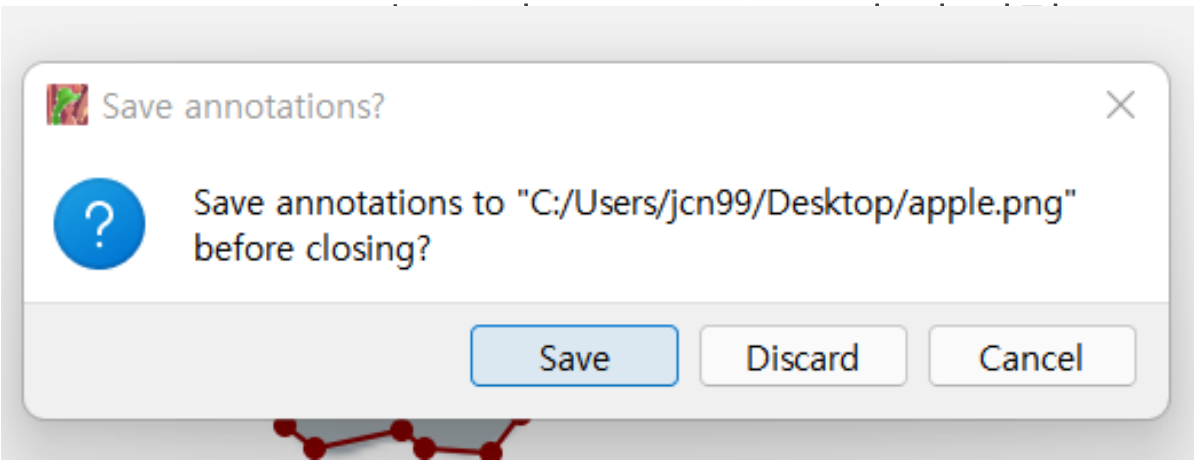
- 세그멘테이션 레이블링:
  - 세그멘테이션 레이블링
  - Opendir -> 다수의 레이블링 해야될 데이터 있는 경우에 사용
  - Create Polygons 클릭 후 + 변하면 해당 부분 포인트에 클릭
  - 확대해서 촘촘히 Point 클릭할 것, 대상 클래스에 맞게 설정

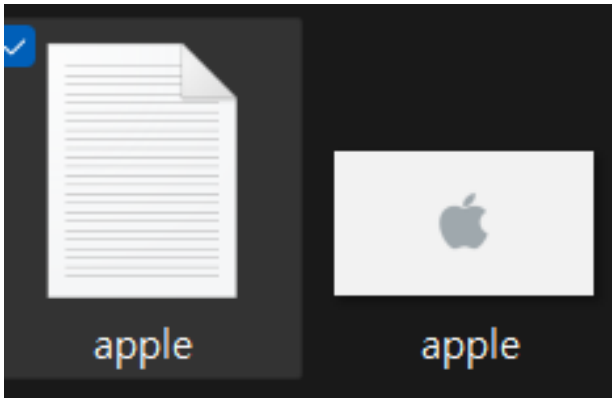




어노테이션 툴 (10/14)

- 세그멘테이션 레이블링:
  - 세그멘테이션 레이블링
  - Next image 통해 다음 이미지
  - Prev Image 통해 이전 이미지
  - Edit Poly gons 통해 잘못된 포인트 수정





어노테이션 툴 (11/14)

- 세그멘테이션 레이블링:
  - Annotation json 결과 값


```
{
  "version": "5.0.1",
  "flags": {},
  "shapes": [
    {
      "label": "apple",
      "points": [
        [
          156.75675675675674,
          129.1261261261261
        ],
        [
          150.9009009009009,
          124.62162162162161
        ],
```

```
        [
          173.87387387387386,
          125.52252252252251
        ]
      ],
      "group_id": null,
      "shape_type": "polygon",
      "flags": {}
    }
  ],
  "imagePath": "apple.png",
  "imageData": "iVBORw0KGgoAAAANSUhEUgAAAVkAAADBCAYAAABlsMNBAAAOXElEQVR4nO3daW8bR4KH8X/1
H4v1ILLYKOf9gY57Z/K9TyzHslIpjrTf6Wh7q6UoCu9ngMAvEFlslPFkqtHocrGa4CM8z6i91dl/f/O1QjJf5Jf/dn+1++6fV
wE29+6WpF//dn+1++6fV5JT0n625gsFNsky7ZnqV6p6B///FSd9jYrEXAn3E
K2k+z/T8xUud9s7XPRwUCE9Gwl0Zs1gjWymVFEWhklkqU6DtqMYYnfR6KseRoihUq1Ff95BQAEQWt1arVVUplTSbJoV
b7U0n2eyVnpzfLr4oPn5rNZPsdZez+saY+T5voLAvzpu0CpJ5sqy7Hre1xjvdp8/X3zeerWiB4d7erjfVRzH3OzCShFZOLN
  "imageHeight": 193,
  "imageWidth": 345
}
```

어노테이션 툴 (12/14)

- 세그멘테이션 레이블링:
  - Json 결과 예시


```
{
"image_name": "358.jpg",
"dough_area": 274627.0,
"dough_area_per": 0,
"dough_cnt": 1,
"dough_label": "dough",
"dough_similarity_measure": 5.707910296861916e-08,
"dough_org_coordi": [[502, 313], [1261, 125], [613, 759]],
"total_score": 0
}
```


  - Mask 결과 예시:

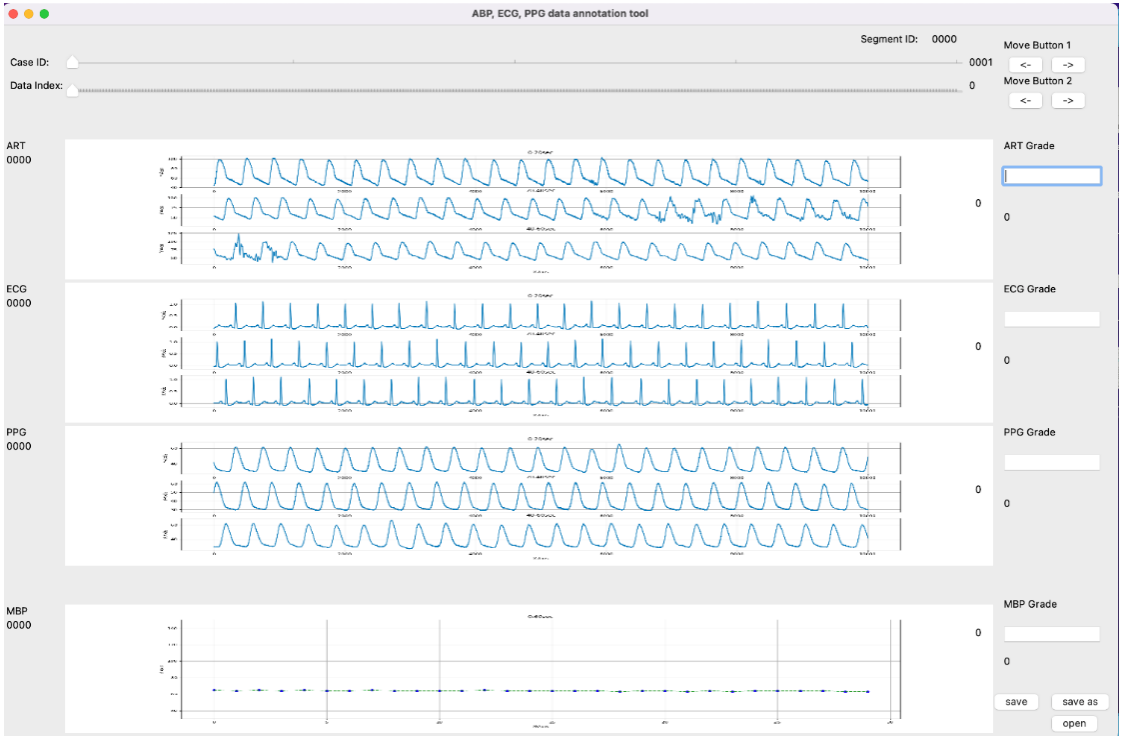


어노테이션 툴 (13/14)

- 실세계에서 Annotation tool:
  - 실제로 Open source로 되어 있지 않은 레이블링 툴은 직접 제작해야 함
  - 기존에 Open source로 되어 있는 것은 모든 레이블링을 다 해결해줄 수는 없음
  - 실세계에서는 다양한 환경에서 각각 다른 조건의 요구사항들이 발생함
  - 이를 검증하기 위해서는 새로운 annotation tool을 만들어서 레이블링을 진행하고
  - 생성 된 학습 데이터를 기반으로 모델을 학습을 진행함

어노테이션 툴 (14/14)

- 실세계에서 Annotation tool:



Signal grading tool

질문 및 정리

